There is nothing special about this index—except we define the `director` field as a `keyword` type, to avoid complexity. Now, let's index a few movies.

Listing 9.7 Indexing three movies

```
PUT classic_movies/_doc/1
{
  "title":"Jaws",
  "director":"Steven Spielberg"
}

PUT classic_movies/_doc/2
{
  "title":"Jaws II",
  "director":"Jeannot Szwarc"
}

PUT classic_movies/_doc/3
{
  "title":"Ready Player One",
  "director":"Steven Spielberg"
}
```

The documents are self-explanatory.

9.3.2 The terms lookup query

Now that we have indexed these three documents, let's return to the `terms` lookup query discussion. Say we wish to fetch all movies directed by Spielberg. However, we don't want to construct a `terms` query and provide the terms directly up front. Instead, we let the `terms` query know to get the values of the terms from a document. The next listing does exactly this.

Listing 9.8 A `terms` lookup search

```
GET classic_movies/_search
{
  "query": {
    "terms": {
      "director": {
        "index":"classic_movies",
        "id":"3",
        "path":"director"
      }
    }
  }
}
```

A terms query (with a twist!)

The field that we are interested in searching against

The index field denotes the name of the index where the document resides.

Name of the field containing the terms for the query

Search field in the current document

This code requires a bit of explanation. We create a `terms` query with `director` as the field in which we want to look for multiple search terms. In a regular `terms` query, we would provide an array with a list of names. But here, we ask the query to look up the `director` values in another document: the document with the `id` value `3`.